

Gradient Descent

DATA 443/643 · Week 1, note 1.3

Week 1

American University

Ahmad Mousavi

Ideas from CS231N L3–L4 and Nielsen Ch. 1. Original slides; those sources are not copied.

Lecture plan

- 1 Loss as a surface

- 2 The update, two steps, overshoot

- 3 Forward, loss, backward, update

- 4 One ReLU unit as gates

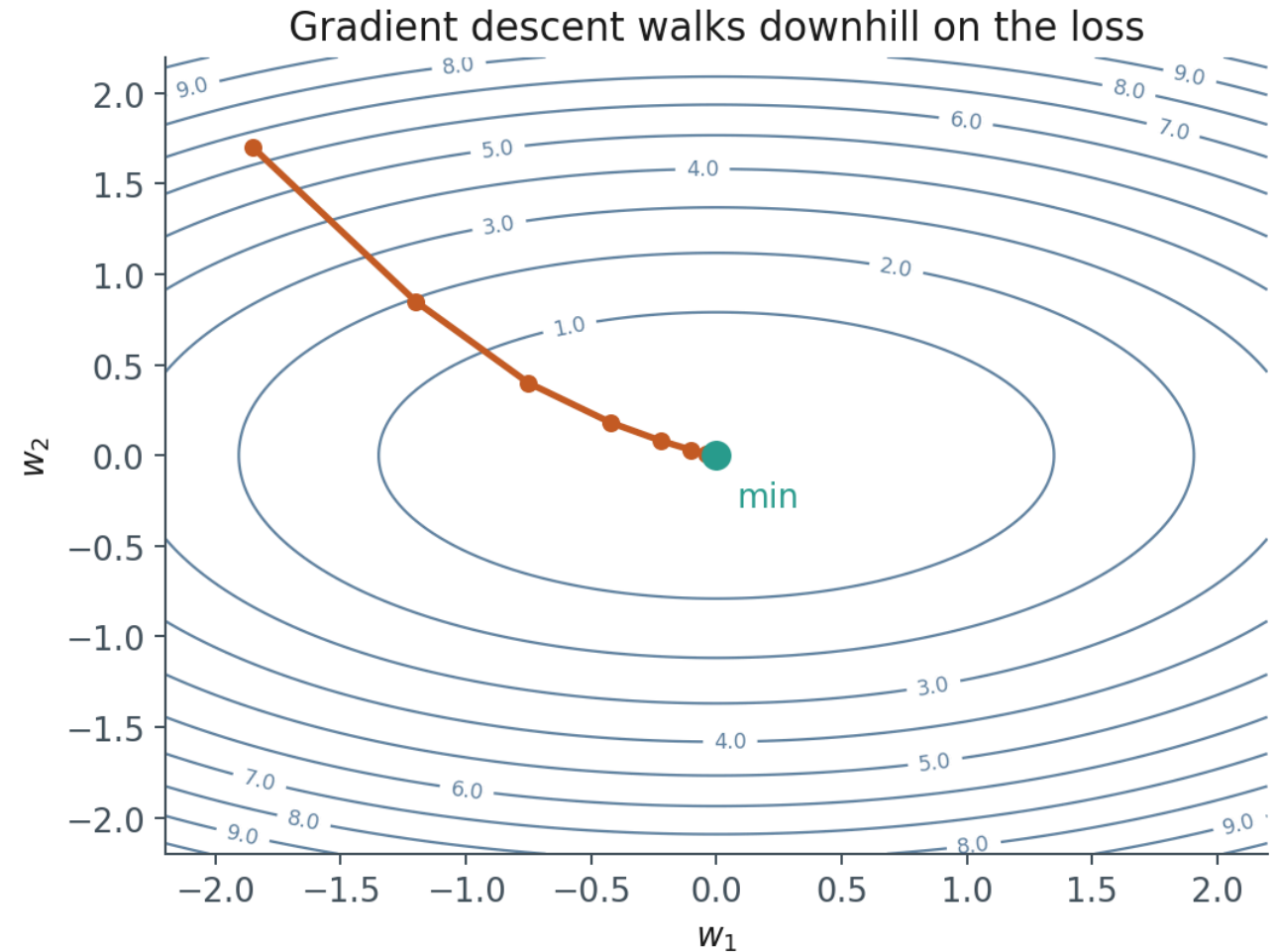
First time: what / why / formula

- What / why. Repeatedly walk downhill on a loss $L(\theta)$. Stochastic GD uses a minibatch, not the full dataset. You cannot solve $\nabla L = 0$ in closed form for a deep net. This is the training loop of pretraining.
- Architecture. Forward (compute L) $\rightarrow$ backward (compute $\nabla_{\theta} L$) $\rightarrow$ update θ .
- Formula. $\theta \leftarrow \theta - \eta \nabla_{\theta} L$. Chain rule on one ReLU: $\partial L / \partial w = (\partial L / \partial a) 1_{z > 0} x$.
- Tradeoffs. + Simple, scales to millions of steps. – Local minima / saddles, needs a step-size choice, noisy minibatches.

How. Pick a step size η . Too large: diverge. Too small: crawl. Adam is a later variant with per-coordinate scales.

Training is walking downhill

- Pack weights into θ . $L(\theta)$ is a scalar.
- Loss = cost = empirical risk.
- The gradient points uphill. Step the other way.



The update

$$\theta \leftarrow \theta - \eta \nabla L(\theta)$$

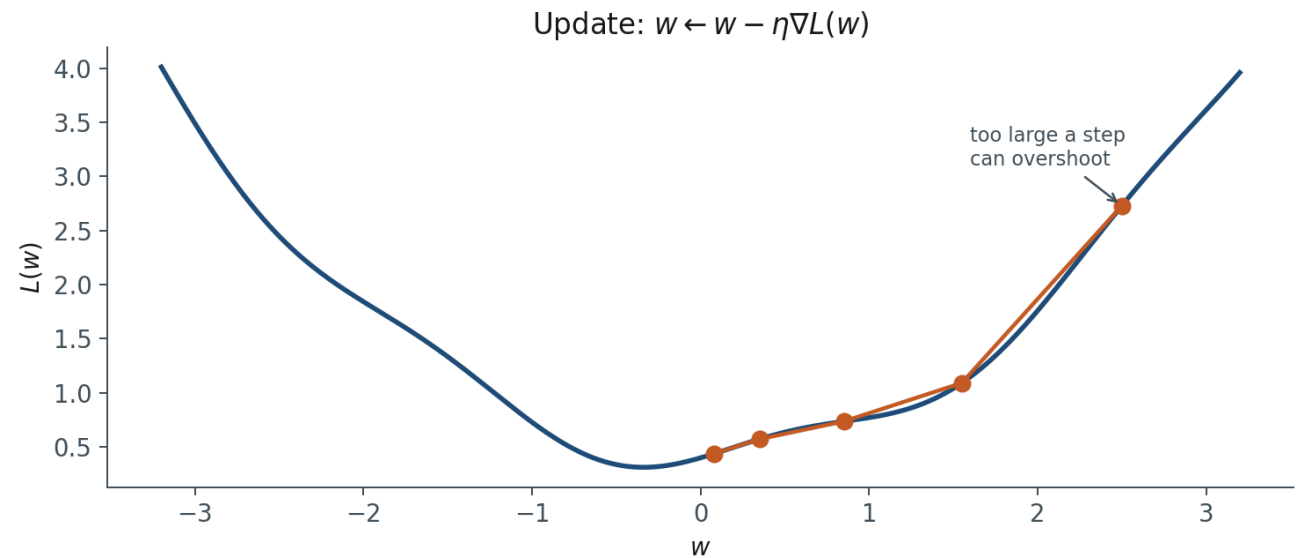
η too small: crawl. Too large: overshoot.

SGD estimates the gradient on a minibatch.

Adam waits. Nielsen Ch. 1 is enough for today.

Two steps, then overshoot

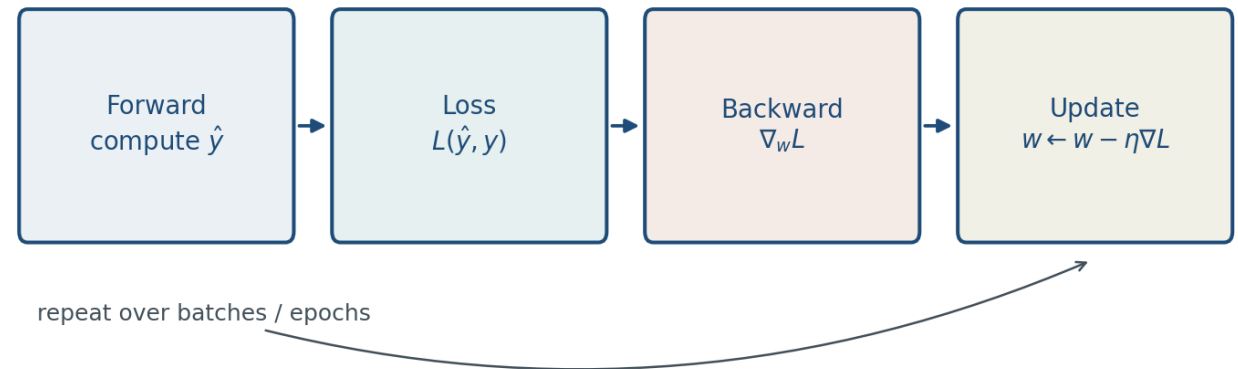
- $L(\theta) = (\theta - 3)^2$, $\theta_0 = 0$, $\eta = 0.25$.
- $\theta_1 = 1.5$, $\theta_2 = 2.25$. Minimum at 3.
- $\eta = 2$ sends $\theta_1 = 12$: overshoot.



Forward, loss, backward, update

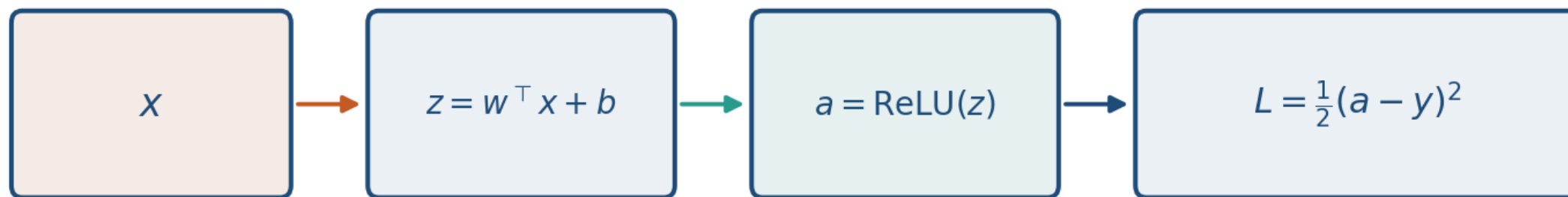
- Keep activations. Backward needs them.
- Autograd is backprop: the chain rule.
- Forget `zero_grad()`: gradients accumulate.

Training a net is this loop. Backprop is just the chain rule.



One ReLU unit as a computational graph

One ReLU unit: cache x and z ; autograd is this chain



backward: $(a - y) \cdot 1_{z > 0} \cdot x$ (CS231N: local $\times$ upstream)

Each op is a gate. Local derivative times upstream.

The chain rule you must write

$$\frac{\partial L}{\partial w} = (a - y) \text{ReLU}'(z) x$$

Squared error on one ReLU neuron.

Softmax/CE cousin:

$$\frac{\partial L}{\partial w} = (p - y)x.$$

$x = 2, w = 0.5, y = 0$:
gradient 2, then $w \leftarrow 0.3$ at
 $\eta = 0.1$.

Lab 1 loss takes logits

Wrong

`sigmoid(z)`, then `BCEWithLogitsLoss`. You squash twice.

Right

Pass `z` into `BCEWithLogitsLoss`. The sigmoid is inside the loss.

Next-token NLL is this same loop at vocabulary scale.